Local Conversation Audit Pipeline

GC1–GC10 Compliance Checker (Offline)

A step–by–step guide for non–technical users on macOS

Project: `crown_capital`

Author: János Horváth

Designed so that no conversation data leaves your machine

---

1 Quick Overview (Plain Language)

What this tool does

This tool reads a long text conversation (for example, a dialogue between a person and an AI assistant) and automatically checks it against ten governance and compliance topics (called **GC1–GC10**).
All analysis happens **locally on your computer**. Nothing is sent to the cloud.

You provide:

- A single text file called `conversation.txt`, containing the full conversation.

The tool:

- Splits the conversation into many small overlapping pieces (*chunks*).
- Sends each chunk to an AI model running locally via **Ollama**.
- For each chunk, asks the model the same 10 questions (GC1–GC10).
- Saves structured results and summaries into simple text and JSON files.

Who is this for?

This guide is written so that someone with *no programming background* can follow it, step by step, on a macOS machine. If you can open the Terminal and copy–paste commands, you can run this tool.

2 Files in Your Folder

All files live in a single folder called `crown_capital`. Inside that folder you have:

```
crown_capital/
  chunk_qa_local_llm.py   # main Python program
  run_chunk_qa.sh         # start script (you will run this)
  conversation.txt        # your input conversation (plain text)
  output.jsonl            # per-chunk audit results  (created by the tool)
  output_summary.txt      # human-readable summary   (created by the tool)
  output_rollup.json      # GC statistics            (created by the tool)
```

```
output_meta.json        # metadata about the run   (created by the tool)
```

At the beginning you will usually only have:

- `chunk_qa_local_llm.py`
- `run_chunk_qa.sh`
- `conversation.txt`

The four `output*` files are created after the first successful run.

3 Before You Begin: A Few Simple Concepts

3.1 The Terminal (Your Command Window)

Opening the Terminal

1. Press **Command** + **Space** to open Spotlight search.
2. Type `Terminal`.
3. Press **Enter**.

When this guide says "run this command", it means:

> *Copy the command, paste it into the Terminal, and press **Enter**.*

3.2 Important: Lines Starting With # Are Comments

In this document you will see examples like:

```
# Example usage (DO NOT TYPE THIS LINE)
./run_chunk_qa.sh conversation.txt output.jsonl ...
```

Common mistake: `zsh: command not found: #`

If you copy *both* lines above into Terminal, you will see:
`zsh: command not found: #`
This is harmless. It just means you tried to run a comment as a command.
Always run only the actual command lines, not the lines starting with #.

4 What Software You Need

You will need the following on your Mac:

- **Homebrew** — a package manager.
- **Python 3** — the language in which the tool is written.
- **pip** — Python's package installer.
- **Python virtual environment** (`venv`) — an isolated Python setup inside the folder.
- **Ollama** — runs the AI model locally.
- The **model** `llama3` — the specific AI model used.

The next sections walk you through checking and installing each one.

5 Step 1: Install and Check Homebrew

5.1 Check if Homebrew Is Already Installed

Step 1.1. In Terminal, type:

```
brew --version
```

- If you see something like `Homebrew 4.x.y` ✓ Homebrew is installed. Move on to Step 2.
- If you see `command not found: brew` ✗ you need to install Homebrew.

5.2 Install Homebrew (Only If Needed)

Step 1.2. In Terminal, run the official installer:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Follow the on-screen instructions. At the end:

- The installer may show a line such as `echo 'export PATH=...'`.
- Copy that line, paste it into Terminal, and press **Enter**.
- Close the Terminal window and open a new one.

Step 1.3. Check again:

```
brew --version
```

Once you see a version number, Homebrew is ready.

6 Step 2: Install and Check Python 3

6.1 Check Python 3

Step 2.1. Run:

```
python3 --version
```

- If you see `Python 3.x.y` ✓ Python 3 is installed.
- If you see an error or only Python 2, install Python 3.

6.2 Install Python 3 (If Needed)

Step 2.2. In Terminal:

```
brew install python
```

When finished, open a new Terminal window and run again:

```
python3 --version
```

6.3 Check pip

Step 2.3. Run:

```
python3 -m pip --version
```

You should see something like:

```
pip 25.3 from ... (python 3.x)
```

If not, reinstall Python using Homebrew.

7 Step 3: Python Virtual Environments (venv)

Step 3.1. Check that `venv` is available:

```
python3 -m venv --help
```

If help text appears, your system is ready. The script will create and manage the virtual environment automatically.

8 Step 4: Install and Check Ollama

8.1 Install Ollama

Step 4.1. In Terminal:

```
brew install ollama
```

8.2 Check Ollama Command

Step 4.2. Run:

```
ollama --version
```

If you see something like `ollama 0.12.11`, the command is recognized.

8.3 Start or Restart the Ollama Service

Step 4.3. Start Ollama as a background service:

```
brew services start ollama
```

If you later need to restart it:

```
brew services restart ollama
```

You may see messages such as:

```
Service ‘ollama‘ already started, use ‘brew services restart ollama‘ to restart.
```

8.4 Download the llama3 Model

Step 4.4. Fetch the model (first time only):

```
ollama pull llama3
```

This can take several minutes and download several gigabytes.

8.5 Verify the Model Is Installed

Step 4.5. List models:

```
ollama list
```

You want to see something like:

```
NAME             ID              SIZE      MODIFIED
llama3:latest    365c0bd3c000    4.7 GB    12 seconds ago
```

If `llama3` appears, Ollama is ready to use.

9 Step 5: Prepare the Project Folder

Step 5.1. Choose a location for the folder (for example, your Desktop).

Step 5.2. Create a folder named `crown_capital`.

Step 5.3. Place **three** files into that folder:

- `chunk_qa_local_llm.py`
- `run_chunk_qa.sh`
- `conversation.txt`

Your folder should now look like:

```
crown_capital/
  chunk_qa_local_llm.py
  run_chunk_qa.sh
  conversation.txt
```

About `conversation.txt`

- This should be a plain text file (UTF–8 is standard).
- Copy and paste your conversation into this file.
- Long conversations are fine; the script will split them into small pieces internally.

10 Step 6: Make the Start Script Executable

Step 6.1. In Terminal, change into the project folder. For example, if it's on your Desktop:

```
cd ~/Desktop/crown_capital
```

Step 6.2. Make `run_chunk_qa.sh` executable (you only do this once per folder):

```
chmod +x run_chunk_qa.sh
```

If there is no error message, this step is complete.

11 Step 7: Run the Pipeline (Recommended Method)

Now you can run the full process with one command.

Step 7.1. From inside `crown_capital`:

```
./run_chunk_qa.sh conversation.txt output.jsonl \
  --chunk-size 1500 \
  --overlap 300 \
  --model-name llama3
```

Meaning of each part:

- `./run_chunk_qa.sh` — runs the helper script.
- `conversation.txt` — input file.
- `output.jsonl` — main output file (will be created).
- `-chunk-size 1500` — length of each chunk (in characters).
- `-overlap 300` — how much chunks overlap (in characters).
- `-model-name llama3` — which local model to use in Ollama.

11.1 What the Script Does Internally

- Checks that `python3` is available.
- Checks that `ollama` is installed and responding.
- Creates a virtual environment (.venv) inside `crown_capital` if needed.
- Activates the virtual environment.
- Upgrades `pip`.
- Installs extra Python packages if a `requirements.txt` file is present.
- Runs `chunk_qa_local_llm.py` with the arguments you provided.

11.2 Example Output While Running

You may see output similar to:

```
Creating virtual environment in ./crown_capital/.venv...
Successfully installed pip-25.3
No requirements.txt found, skipping dependency installation.
Running ./chunk_qa_local_llm.py with args: conversation.txt output.jsonl \
  --chunk-size 1500 --overlap 300 --model-name llama3
Reading conversation...
Creating overlapping chunks...
Created 906 chunks in total.
Will process 906 chunk(s) (from index >= 0, max_chunks=None).
Using OllamaBackend with model 'llama3'.
Chunks to process in this run: 906

=== Processing chunk 0 (0-1500) [1/906] ===
=== Processing chunk 1 (1200-2700) [2/906] ===
...
```

When processing finishes, the script computes summaries and roll-up statistics and exits.

12 Step 8: Understanding the Output Files

After a successful run, your folder will contain:

```
crown_capital/
  chunk_qa_local_llm.py
  run_chunk_qa.sh
  conversation.txt
  output.jsonl
  output_summary.txt
  output_rollup.json
  output_meta.json
  .venv/              # (hidden virtual environment folder)
```

12.1 `output.jsonl` (Detailed Results)

- File type: JSON Lines (one JSON object per line).
- Each line corresponds to one chunk of the conversation.
- For each chunk it contains:
  - The chunk index and character range.
  - For each GC code (GC1–GC10): the value (`TRUE`, `FALSE`, `INSUFFICIENT` or `UNPARSEABLE`) and a short piece of evidence.

12.2 `output_summary.txt` (Human–Readable Summary)

- Plain text summary.
- Easy to read without any technical tools.
- Organized chunk by chunk, listing all ten GC codes and their outcomes.

12.3 `output_rollup.json` (Statistics)

- Aggregated statistics across all chunks.
- For each GC code (GC1–GC10), it shows:
  - How many chunks had value `TRUE`, `FALSE`, `INSUFFICIENT`, `UNPARSEABLE`.
  - A "pass rate" (TRUE divided by all valid responses).

12.4 `output_meta.json` (Metadata)

- Contains information about:
  - When the run was started.
  - Which model was used (e.g. `llama3`).
  - Which input file (`conversation.txt`) was analysed.
  - What command-line options were used.
- Useful for audits and record-keeping.

13 Optional: Running `chunk_qa_local_llm.py` Directly

If you prefer to control each step manually:

13.1 Create and Activate the Virtual Environment

```
cd ~/Desktop/crown_capital   # or your actual path
```

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
```

If you have a `requirements.txt` file:

```
pip install -r requirements.txt
```

13.2 Run the Python Program

```
python chunk_qa_local_llm.py \
  conversation.txt \
  output.jsonl \
  --chunk-size 1500 \
  --overlap 300 \
  --model-name llama3
```

This produces the same outputs as the shell script.

14 Troubleshooting (Frequently Seen Issues)

14.1 `zsh: command not found: #`

Cause

You tried to run a comment line (starting with `#`) as a command.

Fix: Ignore lines starting with `#`. Only run the command lines below them.

14.2 Ollama "server not responding"

If you see errors like:

```
Error: ollama server not responding - could not find ollama app
```

1. Check that `ollama` is installed:

   ```
   ollama --version
   ```

2. Start or restart the service:

   ```
   brew services start ollama
   # or
   brew services restart ollama
   ```

3. Verify that it lists the model:

   ```
   ollama list
   ```

14.3 File Not Found (`conversation.txt`)

If the script says the conversation file is missing:

- Make sure you are in the correct folder:
  `cd ~/Desktop/crown_capital  # adjust path as needed`

- Confirm that `conversation.txt` is actually in that folder.

14.4 Virtual Environment Issues

If the environment seems broken (for example, missing modules):

1. Delete the virtual environment folder:

   ```
   rm -rf .venv
   ```

2. Run the helper script again:

   ```
   ./run_chunk_qa.sh conversation.txt output.jsonl \
     --chunk-size 1500 \
     --overlap 300 \
     --model-name llama3
   ```

This recreates a clean virtual environment.

15 Final Recap

Checklist for a successful run

- ✓ Homebrew installed and working (`brew -version`).
- ✓ Python 3 installed (`python3 -version`).
- ✓ pip available (`python3 -m pip -version`).
- ✓ Ollama installed and running (`ollama -version`, `brew services start ollama`).
- ✓ `llama3` model pulled (`ollama pull llama3`, visible in `ollama list`).
- ✓ Folder `crown_capital` contains:
  - `chunk_qa_local_llm.py`
  - `run_chunk_qa.sh`
  - `conversation.txt`
- ✓ Script made executable:

```
chmod +x run_chunk_qa.sh
```

- ✓ Run command executed:

```
./run_chunk_qa.sh conversation.txt output.jsonl \
  --chunk-size 1500 \
  --overlap 300 \
  --model-name llama3
```

16 Step 9: Optional Export to TAIGA Metrics (Taiga Integration)

The local GC pipeline already gives you detailed per–chunk results and aggregated GC statistics. In many organizations, however, you will want to feed those results into a broader governance or risk dashboard.

The **Taiga export** step does exactly that: it converts `output_rollup.json` (the GC rollup) into a structured JSON payload of **TAIGA metrics** that can be consumed by downstream governance tools (such as a Taiga board or a central risk registry).

High–level idea

The Taiga export step:

- Takes only the **aggregated** GC results as input (`output_rollup.json`).
- Computes **per–GC risk bands** and several composite **TAIGA metrics** (ADR, JES, OI, BFS, CRE).
- Produces a single machine–readable JSON file that can be ingested by your Taiga integration or other governance tooling.

No raw conversation text is used at this stage — only GC counts and pass rates.

16.1 Additional Helper Tools and Files

In addition to the files used by the GC pipeline, the Taiga export step uses:

- `run_taiga_uv.sh` — a shell script that:
  - Ensures a Python virtual environment exists (via `uv`).
  - Installs any required Python packages.
  - Runs `taiga_system.py` inside that environment.
- `taiga_system.py` — the main Taiga export program, which:
  - Reads `output_rollup.json`.
  - Computes detailed per–GC risk metrics.
  - Derives the TAIGA composite metrics (ADR, JES, OI, BFS, CRE).
  - Optionally uses the same **local** LLM backend as the GC pipeline (via Ollama) to map GCs to TAIGA metrics.
  - Writes a structured JSON snapshot if `-output-json` is provided.

Your folder will eventually look like:

```
crown_capital/
  chunk_qa_local_llm.py
  run_chunk_qa.sh
  run_taiga_uv.sh
  taiga_system.py
  conversation.txt
  output.jsonl
  output_summary.txt
  output_rollup.json
  output_meta.json
  outputs/
    e_corp_taiga_from_gc.json   # example Taiga export (created)
  .venv/
```

What is uv?

`uv` is a fast Python packaging tool that combines environment management and dependency installation. You do *not* need to learn `uv` in detail; the helper script `run_taiga_uv.sh` uses it for you. You only need to ensure that the `uv` command is available on your system.

16.2 Step 9.1: Ensure uv Is Installed

Step 9.1. In Terminal, check whether `uv` is available:

```
uv --version
```

- If you see a version number, you are ready to proceed.
- If you see `command not found: uv`, install `uv` using your preferred method (for example, via Homebrew or the official installer from `https://docs.astral.sh/uv/`).

You only need to do this once per machine.

16.3 Step 9.2: Prepare the Taiga Export Script

Step 9.2. Make sure that `run_taiga_uv.sh` and `taiga_system.py` are in the same `crown_capital` folder as your other files.

Step 9.3. From inside the `crown_capital` folder, make the script executable (only once per folder):

```
cd ~/Desktop/crown_capital    # or your actual path
chmod +x run_taiga_uv.sh
```

If there is no error message, the script is ready to use.

16.4 Step 9.3: Inputs and Outputs of the Taiga Export

Input: `output_rollup.json`

- Produced by the main GC pipeline in Step 8.
- Contains, for each GC code (GC1–GC10):
  - Counts of `TRUE`, `FALSE`, `INSUFFICIENT`, `UNPARSEABLE`.
  - A pass rate for that GC.
- No raw conversation content is present — only aggregated statistics.

Output: TAIGA JSON Snapshot

- A single JSON file whose path you control via the `-output-json` argument (for example, `outputs/e_corp_taiga_from_gc.json`).
- This file includes:
  - Overall governance coverage and risk scores.
  - Per–GC metrics (risk bands, coverage ratios, evidence notes).
  - TAIGA metric summaries (ADR, JES, OI, BFS, CRE).
  - An AGI–style composite risk (if computed).
  - A short list of the "weakest" GCs and high–level governance recommendations.
- Designed to be ingested by automation that creates or updates records in Taiga or another governance platform.

16.5 Step 9.4: Running the Taiga Export

Once you have successfully run the main GC pipeline and `output_rollup.json` exists in the folder, you can invoke the Taiga export with one command.

Example: Export GC Results for Tenant `e_corp`

From inside `crown_capital`, run:

```
bash run_taiga_uv.sh output_rollup.json \
  --tenant-id e_corp \
  --output-json outputs/e_corp_taiga_from_gc.json
```

Meaning of each argument:

- `run_taiga_uv.sh` — helper script that:

  - Creates (or reuses) a virtual environment in `.venv/`.
  - Installs required Python dependencies via `uv`.
  - Runs `taiga_system.py` with the arguments you supply.
- `output_rollup.json` — the GC rollup produced in Step 8.
- `-tenant-id e_corp` — a label for the tenant, client or business unit. You can choose any string (for example, `acme_bank_north_america`).
- `-output-json outputs/e_corp_taiga_from_gc.json` — path to the Taiga JSON snapshot. You can choose a different folder or filename.

If the `outputs/` folder does not yet exist, create it first:

```
mkdir -p outputs
```

Optional: Selecting the Local Model

By default, `taiga_system.py` uses the same local LLM backend (via Ollama) as the GC pipeline. Advanced users can override the model by setting the `TAIGA_LLM_MODEL` environment variable or by passing `-model-name` directly to `taiga_system.py`. In most cases you do not need to change this.

16.6 Step 9.5: Inspecting the Taiga JSON Snapshot

After a successful run, you should see a new file such as:

```
outputs/e_corp_taiga_from_gc.json
```

You can inspect it with any text editor or with `jq` on the command line:

```
jq '.' outputs/e_corp_taiga_from_gc.json | less
```

Conceptually, the JSON snapshot contains:

- A top–level object with fields like:
  - `tenant_id`
  - `governance_coverage_score`
  - `governance_risk_score`
  - `band` (e.g. Low / Medium / High / Critical)
  - `gc_metrics` (array of per–GC records)
  - `taiga_metrics` (ADR, JES, OI, BFS, CRE)
  - `agi` (overall composite if computed)
  - `weakest_gcs` and `recommendations`
- Each GC entry summarises how that GC performed across the conversation and what drove its risk band.
- Each TAIGA metric aggregates the relevant GCs into a single, dashboard–ready score with supporting evidence.

16.7 Step 9.6: How the Taiga Export Connects to the GC Pipeline

The full data flow now looks like this:

1. **Local GC audit (fully on your machine)**
   (a) `conversation.txt` is processed by `chunk_qa_local_llm.py` via `run_chunk_qa.sh`.
   (b) This produces:
      - `output.jsonl` (per–chunk GC results),
      - `output_summary.txt` (human–readable summary),
      - `output_rollup.json` (aggregated GC statistics),
      - `output_meta.json` (run metadata).
2. **Taiga export (aggregated–only view)**
   (a) `run_taiga_uv.sh` reads `output_rollup.json`.
   (b) `taiga_system.py` computes per–GC bands and TAIGA metrics.
   (c) The resulting Taiga JSON snapshot is written to `outputs/e_corp_taiga_from_gc.json` (or your chosen path).
   (d) Your Taiga integration or governance platform imports that JSON to create or update issues, risks, or scorecards.

Why this design?

The Taiga export step is intentionally separated from the main GC pipeline:

- It operates only on aggregated data (`output_rollup.json`), reducing data sensitivity.
- It makes it easy to re–run the export with different tenants, models or output schemas without re–analysing the original conversation.
- It provides a clean, machine–readable interface between local audits and enterprise governance tooling.

Authorship

This pipeline and guide were prepared by **János Horváth**, with the goal of making a governance–friendly, fully local audit workflow accessible even to non–technical users.

Quick command reference: run each part end–to–end

This is the *minimal set of commands* you need to run the full pipeline for a long conversation file and then export the aggregated GC results into a TAIGA–style metrics JSON.

Before you start

- Open Terminal and **change into your project folder**, e.g.:
  `cd ~/Desktop/crown_capital`
- Make sure the helper scripts are executable (you only need to do this once):
  `chmod +x run_chunk_qa.sh`
  `chmod +x run_taiga_uv.sh`
- Ensure your input conversation file exists. In this example we assume:
  `data/conversation_mega_long.txt`
  If your file is named `conversation.txt` and sits directly in `crown`$_{c}apital/, adjustthecommandaccordingly.$

1. Run the local GC audit on a long conversation file

This step:

- Splits the conversation into overlapping chunks.
- Sends each chunk to the local `llama3` model via Ollama.
- Writes detailed per–chunk results to `output.jsonl`.
- Also creates rollup and summary files (`output_rollup.json`, `output_summary.txt`, `output_meta.json`).

Run:

```
bash run_chunk_qa.sh data/conversation_mega_long.txt output.jsonl \
  --chunk-size 1000 \
  --overlap 200 \
  --model-name llama3
```

Key pieces:

- `data/conversation_mega_long.txt` — path to your input file.
- `output.jsonl` — main per–chunk results file (created).
- `-chunk-size 1000` — size of each chunk (characters).
- `-overlap 200` — overlap between neighbouring chunks.
- `-model-name llama3` — local Ollama model to use.

Behind the scenes, `run_chunk_qa.sh` also:

- Checks that `python3` and `ollama` are available.
- Creates (or reuses) a virtual environment in `.venv/`.
- Upgrades `pip` inside that environment.
- Looks for a `requirements.txt` file:
  - In your current setup there is *no* `requirements.txt`, so it simply prints a short message such as "No requirements.txt found, skipping dependency installation."
  - If you later add a `requirements.txt`, the script will automatically install the listed packages.
- Finally calls `chunk_qa_local_llm.py` with the arguments you provided.

2. Export the GC rollup into TAIGA metrics JSON

Once the GC audit has finished, you should have:

- `output_rollup.json` in the project folder.
- An `outputs/` folder (create it if it does not exist):
  `mkdir -p outputs`